

Лабораторная работа №5: Интеграция сверточной нейронной сети в FastApi приложение

Цель работы

Целью данной лабораторной работы является изучение принципов работы и практическая реализация **сверточной нейронной сети (CNN)** для классификации изображений на языке Python с использованием библиотеки **PyTorch** и её интеграция в существующее веб-приложение на базе фреймворка **FastAPI**.

Задачи работы

В ходе выполнения лабораторной работы необходимо:

1. Изучить теоретические основы искусственных нейронных сетей и сверточных нейронных сетей в частности.
2. Освоить работу с фреймворком PyTorch: объекты Dataset, DataLoader, nn.Module, оптимизаторы, функции потерь.
3. Загрузить и подготовить обучающую выборку CIFAR-10.
4. Спроектировать и реализовать архитектуру сверточной нейронной сети.
5. Обучить модель и сохранить её в файл весов.
6. Интегрировать обученную модель в существующее FastAPI-приложение, расширив его новым эндпоинтом классификации.
7. Выполнить задания для самостоятельной работы.
8. Ответить на контрольные вопросы.

Базовый проект

Лабораторная работа выполняется на основе проекта-каркаса, разработанного в предыдущей лабораторной работе:

Репозиторий: <https://github.com/Constantine4002000/fast-api-ip>

Базовый проект реализует трёхуровневую архитектуру FastAPI-приложения для загрузки и хранения изображений в SQLite. Структура каталогов:

```
fast-api-ip/
├── data/           # Уровень данных (SQLite + DB-API)
├── db/             # Файл базы данных pictures.db
├── model/          # Pydantic-модели данных
└── service/        # Бизнес-логика, работа с OpenCV
```

```
|— src/          # Точка входа FastAPI-приложения (main.py)
|— web/          # HTTP-эндпоинты (роутеры)
```

В рамках текущей лабораторной работы данная структура будет **расширена** новыми модулями, связанными с нейросетевой классификацией изображений.

Теоретическая часть

Краткие сведения об искусственных нейронных сетях

Искусственная нейронная сеть (ИНС) — это математическая модель, построенная по принципу организации и функционирования биологических нейронных сетей. Базовой вычислительной единицей сети является искусственный нейрон, выполняющий следующее преобразование:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right),$$

где x_i — входные значения, w_i — синаптические веса, b — смещение (bias), $f(\cdot)$ — нелинейная функция активации, y — выход нейрона.

Совокупность нейронов, соединённых между собой и сгруппированных в слои, образует нейронную сеть. Процесс настройки её параметров (весов и смещений) на обучающей выборке называется **обучением**, а основным алгоритмом является **метод обратного распространения ошибки (backpropagation)** в сочетании с тем или иным вариантом стохастического градиентного спуска.

Сверточные нейронные сети

Идея и предпосылки

Полносвязные нейронные сети плохо подходят для обработки изображений по двум причинам:

- Слишком большое число параметров. Для цветного изображения $224 \times 224 \times 3$ первый полносвязный слой даже с 1000 нейронов содержит более 150 миллионов весов.
- Игнорируется пространственная структура. Полносвязный слой одинаково реагирует на любую перестановку пикселей, тогда как для изображений важна локальная связность и трансляционная инвариантность.

Эти проблемы решает специализированный тип архитектуры — **сверточная нейронная сеть (Convolutional Neural Network, CNN)**, предложенная Я. Лекуном в работах конца 1980-х — начала 1990-х годов (LeNet-5, 1998). Современный «бум» CNN начался после публикации сети **AlexNet** (Крижевский и др., 2012), победившей в соревновании ImageNet с большим отрывом.

Базовые операции CNN

Сверточный слой (Convolutional Layer). Вместо того чтобы соединять каждый пиксель с каждым нейроном, к изображению применяется набор небольших обучаемых фильтров (ядер), скользящих по входному тензору. Операция дискретной двумерной свёртки определяется так:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n),$$

где I — входная карта признаков, K — ядро свёртки (например, 3×3 или 5×5). Результатом применения одного ядра ко входу является **карта признаков** (feature map). На каждом сверточном слое таких ядер обычно несколько десятков или сотен.

Ключевые свойства свёртки:

- **Локальная связность** — каждый выходной нейрон «видит» только небольшой участок входа (рецептивное поле).
- **Разделяемые веса** — одни и те же параметры ядра применяются ко всему изображению, что резко сокращает число параметров и обеспечивает инвариантность к сдвигу.

Функции активации. После свёртки к каждому элементу карты признаков поэлементно применяется нелинейность. Наиболее распространённой в CNN является **ReLU (Rectified Linear Unit)**:

$$\text{ReLU}(x) = \max(0, x).$$

Также используются Leaky ReLU, ELU, GELU и др.

Слой пулинга (Pooling). Уменьшает пространственное разрешение карты признаков, агрегируя значения в окнах фиксированного размера (обычно 2×2). Чаще всего применяется **max-pooling**, выбирающий максимум в окне:

$$y(i, j) = \max_{(m, n) \in W_{ij}} x(m, n).$$

Пулинг снижает вычислительную сложность и обеспечивает устойчивость к небольшим сдвигам.

Полносвязные слои (Fully Connected, FC). В классической схеме CNN на выходе свёрточно-пулинговых блоков карта признаков «вытягивается» в одномерный вектор, после чего следует один или несколько полносвязных слоёв и, наконец, **выходной слой** с числом нейронов, равным количеству классов. К выходу применяется функция **softmax**:

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^C \exp(z_j)}, \quad k = 1, \dots, C,$$

преобразующая логиты z_k в распределение вероятностей по классам.

Регуляризация. Для предотвращения переобучения применяются:

- **Dropout** — случайное «отключение» части нейронов при обучении;
- **Batch Normalization** — нормировка активаций внутри мини-батча, ускоряющая обучение;
- **Аугментация данных** (Data Augmentation) — искусственное расширение обучающей выборки за счёт случайных преобразований изображений (отражения, повороты, кадрирование, изменение яркости и контраста и др.).

Функция потерь и обучение

Для задачи многоклассовой классификации применяется **функция потерь перекрёстной энтропии (Cross-Entropy Loss)**:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^C y_{ik} \log p_{ik},$$

где y_{ik} — индикатор истинного класса (one-hot), p_{ik} — предсказанная вероятность, N — размер мини-батча, C — число классов.

Минимизация $\mathcal{L}$ выполняется одним из вариантов градиентного спуска: **SGD с моментом (Momentum)**, **Adam**, **AdamW**, **RMSprop** и др.

Типовая архитектура CNN

Обобщённая схема классификационной сверточной сети:

Вход ($C \times H \times W$)

- [Conv → BN → ReLU → Conv → BN → ReLU → MaxPool] × N
- Flatten
- [FC → ReLU → Dropout] × M
- FC (выход: число классов)
- Softmax

Известные архитектуры: **LeNet-5**, **AlexNet**, **VGG-16/19**, **GoogLeNet/Inception**, **ResNet**, **DenseNet**, **EfficientNet**. Современные модели (Vision Transformers, ConvNeXt) частично отходят от классической сверточной парадигмы, однако CNN по-прежнему остаются базовым инструментом компьютерного зрения.

Фреймворк PyTorch

PyTorch — это открытая библиотека для глубокого обучения, разработанная компанией Meta AI. Её ключевые особенности:

- Динамический граф вычислений, упрощающий отладку и реализацию нестандартных архитектур.
- Автоматическое дифференцирование (autograd).
- Прозрачное использование GPU через CUDA.
- Богатая экосистема: torchvision (компьютерное зрение), torchaudio, torchtext, Lightning, Hugging Face.

В работе используются следующие модули PyTorch:

Модуль	Назначение
torch.nn	Слои и функции активации
torch.optim	Оптимизаторы
torch.utils.data	Классы Dataset и DataLoader
torchvision.datasets	Готовые наборы данных (CIFAR-10, MNIST и др.)
torchvision.transforms	Преобразования изображений

Обучающая выборка CIFAR-10

CIFAR-10 — классический эталонный набор данных для задач классификации изображений, разработанный А. Крижевским и Дж. Хинтоном в Университете Торонто.

Официальный сайт: <https://www.cs.toronto.edu/~kriz/cifar.html>

Характеристики набора:

- 60 000 цветных изображений размером **32×32 пикселя**;
- 10 сбалансированных классов (по 6 000 изображений на класс): *самолёт, автомобиль, птица, кошка, олень, собака, лягушка, лошадь, корабль, грузовик*;
- Разделение: 50 000 изображений для обучения, 10 000 — для тестирования.

CIFAR-10 удобен для учебных целей: размер изображений невелик, обучение CNN на CPU занимает приемлемое время (несколько десятков минут), а на GPU — несколько минут.

В PyTorch набор автоматически загружается через `torchvision.datasets.CIFAR10` (см. практическую часть).

Практическая часть

Подготовка окружения

Клонируйте базовый репозиторий и перейдите в каталог проекта:

```
git clone https://github.com/Constantine4002000/fast-api-ip.git
cd fast-api-ip
```

Создайте и активируйте виртуальное окружение:

```
python -m venv venv
# Windows:
venv\Scripts\activate
# Linux / macOS:
source venv/bin/activate
```

Установите дополнительные зависимости. Создайте (или дополните) файл `requirements.txt`:

```
fastapi
uvicorn[standard]
python-multipart
opencv-python
numpy
pydantic
torch
torchvision
pillow
```

Важно. На большинстве систем для CPU-версии PyTorch достаточно команды:

```
pip install torch torchvision
```

Для CUDA-версии воспользуйтесь генератором команд на официальном сайте <https://pytorch.org/get-started/locally/>.

Установите все зависимости:

```
pip install -r requirements.txt
```

Расширение структуры проекта

После выполнения лабораторной работы структура проекта будет выглядеть следующим образом (новые файлы помечены символом +):

```
fast-api-ip/
├─ data/
├─ db/
├─ model/
```

```

├── pictures.py
├── classifier.py          # + архитектура CNN (nn.Module)
├── service/
│   ├── pictures.py
│   └── classifier.py      # + сервис: загрузка модели, инференс
├── web/
│   ├── pictures.py
│   └── classifier.py      # + HTTP-эндпоинт /classify
├── src/
│   └── main.py            # + подключение нового роутера
├── ml/                    # + рабочая папка для ML
│   ├── train.py          # + скрипт обучения
│   ├── evaluate.py        # + скрипт оценки на тесте
│   └── weights/
│       └── cifar10_cnn.pth # + сохранённые веса модели
└── requirements.txt

```

Шаг 1. Описание архитектуры CNN

Создайте файл `model/classifier.py` со следующим содержимым:

```

"""
Архитектура сверточной нейронной сети для классификации
изображений CIFAR-10 (10 классов, изображения 32×32×3).
"""

import torch
import torch.nn as nn
import torch.nn.functional as F

CIFAR10_CLASSES = (
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck",
)

class CIFAR10CNN(nn.Module):
    """Простая, но рабочая CNN для CIFAR-10."""

    def __init__(self, num_classes: int = 10) -> None:
        super().__init__()

        # Блок 1: 3 -> 32 каналов, 32×32 -> 16×16
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 32, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(32)

```

```

# Блок 2: 32 -> 64 каналов, 16x16 -> 8x8
self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
self.bn3 = nn.BatchNorm2d(64)
self.conv4 = nn.Conv2d(64, 64, kernel_size=3, padding=1)
self.bn4 = nn.BatchNorm2d(64)

# Блок 3: 64 -> 128 каналов, 8x8 -> 4x4
self.conv5 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
self.bn5 = nn.BatchNorm2d(128)

self.pool = nn.MaxPool2d(2, 2)
self.dropout = nn.Dropout(0.25)

# Полносвязные слои
self.fc1 = nn.Linear(128 * 4 * 4, 256)
self.fc2 = nn.Linear(256, num_classes)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    # Блок 1
    x = F.relu(self.bn1(self.conv1(x)))
    x = F.relu(self.bn2(self.conv2(x)))
    x = self.pool(x)
    x = self.dropout(x)

    # Блок 2
    x = F.relu(self.bn3(self.conv3(x)))
    x = F.relu(self.bn4(self.conv4(x)))
    x = self.pool(x)
    x = self.dropout(x)

    # Блок 3
    x = F.relu(self.bn5(self.conv5(x)))
    x = self.pool(x)
    x = self.dropout(x)

    # Полносвязная часть
    x = torch.flatten(x, start_dim=1)
    x = F.relu(self.fc1(x))
    x = self.dropout(x)
    x = self.fc2(x)
    return x # логиты (без softmax)

```

Пояснения по архитектуре:

- Сеть состоит из трёх свёрточных блоков. Число фильтров возрастает (32 → 64 → 128) по мере уменьшения пространственного размера карт признаков (32 → 16 → 8 → 4) — типичная для CNN практика.
- В каждом блоке используется BatchNorm2d после свёртки и MaxPool2d(2, 2) для понижения

- размерности.
- Dropout (0.25) препятствует переобучению.
 - Метод forward возвращает «сырые» логиты. Softmax применяется в момент подсчёта потерь (CrossEntropyLoss уже включает его) или при необходимости получить вероятности.

Шаг 2. Скрипт обучения модели

Создайте каталог ml/ и файл ml/train.py:

```
"""
Скрипт обучения CNN на CIFAR-10.
Запуск: python -m ml.train
"""

from pathlib import Path

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
import torchvision
import torchvision.transforms as T

from model.classifier import CIFAR10CNN

# --- Гиперпараметры -----
BATCH_SIZE = 128
NUM_EPOCHS = 15
LEARNING_RATE = 1e-3
WEIGHT_DECAY = 1e-4
DATA_DIR = "./ml/data"
WEIGHTS_PATH = Path("./ml/weights/cifar10_cnn.pth")

def get_dataloaders() -> tuple[DataLoader, DataLoader]:
    """Готовит обучающую и тестовую выборки CIFAR-10."""
    # Аугментации применяются только к обучающей выборке
    train_tf = T.Compose([
        T.RandomCrop(32, padding=4),
        T.RandomHorizontalFlip(),
        T.ToTensor(),
        T.Normalize(mean=(0.4914, 0.4822, 0.4465),
                     std=(0.2470, 0.2435, 0.2616)),
    ])
    test_tf = T.Compose([
        T.ToTensor(),
        T.Normalize(mean=(0.4914, 0.4822, 0.4465),
```



```

        std=(0.2470, 0.2435, 0.2616)),
    ])

    train_set = torchvision.datasets.CIFAR10(
        root=DATA_DIR, train=True, download=True, transform=train_tf,
    )
    test_set = torchvision.datasets.CIFAR10(
        root=DATA_DIR, train=False, download=True, transform=test_tf,
    )

    train_loader = DataLoader(train_set, batch_size=BATCH_SIZE,
                              shuffle=True, num_workers=2)
    test_loader = DataLoader(test_set, batch_size=BATCH_SIZE,
                              shuffle=False, num_workers=2)
    return train_loader, test_loader

def train_one_epoch(model, loader, criterion, optimizer, device) -> float:
    model.train()
    running_loss = 0.0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)
    return running_loss / len(loader.dataset)

@torch.no_grad()
def evaluate(model, loader, criterion, device) -> tuple[float, float]:
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)

        total_loss += loss.item() * images.size(0)
        _, predicted = outputs.max(1)
        correct += predicted.eq(labels).sum().item()
    total += labels.size(0)

```

```

    return total_loss / total, correct / total

def main() -> None:
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"[i] Используется устройство: {device}")

    train_loader, test_loader = get_dataloaders()

    model = CIFAR10CNN(num_classes=10).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(),
                             lr=LEARNING_RATE,
                             weight_decay=WEIGHT_DECAY)
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)

    best_acc = 0.0
    WEIGHTS_PATH.parent.mkdir(parents=True, exist_ok=True)

    for epoch in range(1, NUM_EPOCHS + 1):
        train_loss = train_one_epoch(model, train_loader,
                                      criterion, optimizer, device)
        val_loss, val_acc = evaluate(model, test_loader, criterion, device)
        scheduler.step()

        print(f"Epoch {epoch:02d}/{NUM_EPOCHS} | "
              f"train_loss={train_loss:.4f} | "
              f"val_loss={val_loss:.4f} | "
              f"val_acc={val_acc * 100:.2f}%")

        # Сохраняем лучшие веса
        if val_acc > best_acc:
            best_acc = val_acc
            torch.save(model.state_dict(), WEIGHTS_PATH)
            print(f"    ↳ сохранены веса (acc={best_acc * 100:.2f}%)")

    print(f"[OK] Обучение завершено. Лучшая точность: {best_acc * 100:.2f}%")
    print(f"[OK] Веса сохранены в: {WEIGHTS_PATH.resolve()}")

if __name__ == "__main__":
    main()

```

Пояснения к скрипту обучения:

1. **Аугментация.** Случайное обрезание `RandomCrop(32, padding=4)` и горизонтальное отражение `RandomHorizontalFlip()` применяются только к обучающей выборке — это стандартный набор для CIFAR-10.

2. **Нормировка.** Тензоры приводятся к нулевому среднему и единичной дисперсии по каждому каналу (значения констант — это среднеканальные среднее и СКО, рассчитанные для CIFAR-10).
3. **Цикл обучения** реализует классический паттерн: `zero_grad` → `forward` → `loss` → `backward` → `step`.
4. **Планировщик шага** StepLR уменьшает скорость обучения вдвое каждые 5 эпох, что обычно повышает финальную точность.
5. **Сохранение лучших весов.** Сохраняется только `state_dict` (рекомендованный способ по сравнению с сохранением всего объекта модели).

Запустите обучение:

```
python -m ml.train
```

При обучении на CPU 15 эпох занимают ориентировочно 30–60 минут. На современном GPU — 3–7 минут. Ожидаемая точность на тесте: **80–85 %**.

Шаг 3. Сервисный слой для инференса

Создайте файл `service/classifier.py`:

```
"""
Сервис нейросетевой классификации изображений.
Загружает обученную модель и предоставляет метод predict().
"""

from pathlib import Path
from typing import Final

import cv2
import numpy as np
import torch
import torch.nn.functional as F
import torchvision.transforms as T
from PIL import Image

from model.classifier import CIFAR10CNN, CIFAR10_CLASSES

WEIGHTS_PATH: Final[Path] = Path("ml/weights/cifar10_cnn.pth")
DEVICE: Final[torch.device] = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

_inference_tf = T.Compose([
    T.Resize((32, 32)),
    T.ToTensor(),
    T.Normalize(mean=(0.4914, 0.4822, 0.4465),
                std=(0.2470, 0.2435, 0.2616)),
])
```

```

class Classifier:
    """Singleton-обёртка над обученной CNN."""

    _instance: "Classifier | None" = None

    def __new__(cls) -> "Classifier":
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._init_model()
        return cls._instance

    def _init_model(self) -> None:
        if not WEIGHTS_PATH.exists():
            raise FileNotFoundError(
                f"Файл весов не найден: {WEIGHTS_PATH}. "
                f"Запустите обучение: python -m ml.train"
            )
        self.model = CIFAR10CNN(num_classes=10).to(DEVICE)
        self.model.load_state_dict(
            torch.load(WEIGHTS_PATH, map_location=DEVICE),
        )
        self.model.eval()

    @torch.no_grad()
    def predict(self, img_bgr: np.ndarray, top_k: int = 3) -> list[dict]:
        """
        Возвращает top-k предсказаний для входного изображения BGR.

        :param img_bgr: ndarray формата BGR (как из cv2.imdecode).
        :param top_k: число возвращаемых классов.
        :return: список словарей вида
            [{"label": "...", "probability": 0.87}, ...].
        """
        # OpenCV отдаёт BGR, PyTorch/PIL работают в RGB
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
        pil_img = Image.fromarray(img_rgb)

        tensor = _inference_tf(pil_img).unsqueeze(0).to(DEVICE)
        logits = self.model(tensor)
        probs = F.softmax(logits, dim=1).squeeze(0)

        top_probs, top_idx = torch.topk(probs, k=top_k)
        return [
            {
                "label": CIFAR10_CLASSES[int(idx)],

```

```

        "probability": float(prob),
    }
    for prob, idx in zip(top_probs, top_idx)
]

```

Ключевые моменты:

- **Паттерн Singleton.** Модель загружается в память один раз — при первом обращении. Это критично, так как FastAPI создаёт сервисы при каждом запросе.
- **model.eval().** Переводит модель в режим инференса: отключает Dropout и переводит BatchNorm в режим использования бегущих средних.
- **torch.no_grad().** Отключает построение графа вычислений и расчёт градиентов, экономя память и время.
- **Согласование цветовых пространств.** cv2 использует BGR, а torchvision/PIL — RGB. Перепутать их — типичная ошибка, ведущая к падению точности.

Шаг 4. HTTP-эндпоинт классификации

Создайте файл web/classifier.py:

```

"""
Маршруты FastAPI для классификации изображений.
"""

import cv2
import numpy as np
from fastapi import APIRouter, File, HTTPException, UploadFile

from service.classifier import Classifier

router = APIRouter(prefix="/classify", tags=["classifier"])

@router.post("/")
async def classify_image(
    file: UploadFile = File(...),
    top_k: int = 3,
) -> dict:
    """Принимает изображение, возвращает top-k классов CIFAR-10."""
    if not file.content_type or not file.content_type.startswith("image/"):
        raise HTTPException(
            status_code=400,
            detail="Поддерживаются только файлы изображений (image/*).")
    )
    if not 1 <= top_k <= 10:
        raise HTTPException(
            status_code=400,
            detail="Параметр top_k должен быть в диапазоне 1..10.",
        )

```

```

raw = await file.read()
arr = np.frombuffer(raw, dtype=np.uint8)
img_bgr = cv2.imdecode(arr, cv2.IMREAD_COLOR)
if img_bgr is None:
    raise HTTPException(
        status_code=400,
        detail="Не удалось декодировать изображение.",
    )

try:
    predictions = Classifier().predict(img_bgr, top_k=top_k)
except FileNotFoundError as exc:
    raise HTTPException(status_code=503, detail=str(exc)) from exc

return {
    "filename": file.filename,
    "top_k": top_k,
    "predictions": predictions,
}

```

Шаг 5. Подключение роутера в src/main.py

Откройте src/main.py и добавьте импорт и подключение нового роутера. Минимально измененный файл может выглядеть так:

```

from fastapi import FastAPI

from web import pictures as pictures_router
from web import classifier as classifier_router

app = FastAPI(
    title="FastAPI Image Processing & Classification",
    version="2.0.0",
)

app.include_router(pictures_router.router)
app.include_router(classifier_router.router)

@app.get("/")
def root() -> dict:
    return {"status": "ok", "service": "fast-api-ip"}

```

Шаг 6. Запуск и тестирование

Запустите сервер:

```
uvicorn src.main:app --reload
```

Откройте Swagger UI: <http://127.0.0.1:8000/docs>. В списке должен появиться раздел **classifier** с эндпоинтом POST /classify/.

Для проверки из командной строки выполните:

```
curl -X POST "http://127.0.0.1:8000/classify/?top_k=3" \
  -H "accept: application/json" \
  -F "file=@/path/to/your/image.jpg"
```

Ожидаемый ответ:

```
{
  "filename": "cat.jpg",
  "top_k": 3,
  "predictions": [
    {"label": "cat",    "probability": 0.8721},
    {"label": "dog",    "probability": 0.0814},
    {"label": "deer",   "probability": 0.0231}
  ]
}
```

Шаг 7. Скрипт оценки на тестовой выборке

Дополнительно создайте файл ml/evaluate.py, позволяющий проверить точность сохранённой модели на полной тестовой выборке CIFAR-10:

```
"""
Оценка качества сохранённой модели на тесте CIFAR-10.
Запуск: python -m ml.evaluate
"""

import torch
import torch.nn as nn
from torch.utils.data import DataLoader
import torchvision
import torchvision.transforms as T

from model.classifier import CIFAR10CNN
from service.classifier import WEIGHTS_PATH

def main() -> None:
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    test_tf = T.Compose([
        T.ToTensor(),
        T.Normalize(mean=(0.4914, 0.4822, 0.4465),
                     std=(0.2470, 0.2435, 0.2616)),
    ])

    # ... (rest of the script)
```

```

test_set = torchvision.datasets.CIFAR10(
    root="./ml/data", train=False, download=True, transform=test_tf,
)
test_loader = DataLoader(test_set, batch_size=256, shuffle=False)

model = CIFAR10CNN(num_classes=10).to(device)
model.load_state_dict(torch.load(WEIGHTS_PATH, map_location=device))
model.eval()

criterion = nn.CrossEntropyLoss()
total_loss, correct, total = 0.0, 0, 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)

        total_loss += loss.item() * images.size(0)
        _, predicted = outputs.max(1)
        correct += predicted.eq(labels).sum().item()
        total += labels.size(0)

print(f"Test loss:      {total_loss / total:.4f}")
print(f"Test accuracy: {correct / total * 100:.2f}%")

if __name__ == "__main__":
    main()

```

Задания для самостоятельного выполнения

Каждое задание выполняется как самостоятельная подзадача. По итогам формируется единый отчёт с описанием изменений, скриншотами Swagger UI и графиками (по необходимости).

Задание 1. Обогащение API классификации

Расширьте endpoint `POST /classify/`:

1. Добавьте параметр `save: bool = False`. Если `save=True`, изображение, его имя и предсказанная метка должны сохраняться в существующую таблицу `pictures` (через `service.pictures`), а в поле `description` — записываться JSON-строка с top-1 классом и его вероятностью.
2. Реализуйте новый endpoint `GET /classify/history`, возвращающий все сохранённые предсказания с пагинацией (`limit, offset`).

Задание 2. Эксперимент с архитектурой

Реализуйте альтернативную архитектуру CIFAR10CNNv2 с обоснованием выбора. Минимальные требования:

- не менее 4 свёрточных блоков;
- использование `nn.AdaptiveAvgPool2d`;
- общее количество параметров не более 1 млн (вычислите программно).

Обучите обе модели (v1 и v2) на одинаковых гиперпараметрах и сравните точности. Постройте графики `train_loss` / `val_loss` / `val_accuracy` по эпохам с помощью `matplotlib`. Включите графики в отчёт.

Задание 3. Использование предобученной модели (Transfer Learning)

С помощью `torchvision.models.resnet18(weights=ResNet18_Weights.DEFAULT)`:

1. Замените последний полносвязный слой на новый, рассчитанный на 10 классов CIFAR-10.
2. «Заморозьте» веса всех слоёв, кроме последнего блока и выходного.
3. Дообучите модель в течение 5 эпох.
4. Сравните точность с вашей обычной CNN из практической части.

Задание 4. Замена набора данных

Повторите обучение и интеграцию для одного из наборов:

- **Fashion-MNIST** (`torchvision.datasets.FashionMNIST`, 10 классов одежды) — простейший вариант;
- **CIFAR-100** — более сложная задача (100 классов);
- **STL-10** — изображения 96×96, 10 классов.

Адаптируйте архитектуру (число каналов входа, размер Linear-слоёв, число выходов) и приведите итоговую точность.

Задание 5. Анализ ошибок

Реализуйте скрипт `ml/confusion_matrix.py`, который:

-
1. Прогоняет тестовую выборку через обученную модель.
 2. Строит матрицу ошибок (используйте `sklearn.metrics.confusion_matrix` и `seaborn.heatmap`).
 3. Выводит названия трёх пар классов, которые модель путает чаще всего, и содержательно объясняет причины (например, *cat* $\leftrightarrow$ *dog*).

Задание 6. Учёт устройства запуска

Доработайте `service/classifier.py` так, чтобы:

- при наличии CUDA модель загружалась на GPU,
- эндпоинт `GET /classify/info` возвращал JSON с устройством, числом параметров модели и временем последнего инференса в миллисекундах.

Контрольные вопросы

1. В чём принципиальные отличия сверточной нейронной сети от полносвязной? Каковы преимущества CNN при работе с изображениями?
2. Что такое рецептивное поле нейрона свёрточного слоя? Как изменяется размер рецептивного поля по мере углубления в сеть?
3. Поясните операции **stride** и **padding**. Как они влияют на размер выходной карты признаков?
4. Запишите формулу размерности выхода свёрточного слоя при заданных параметрах входа $H \times W$, размере ядра k , отступе p и шаге s .
5. Зачем в CNN применяются слои пулинга? Сравните max-pooling и average-pooling.
6. Что такое **Batch Normalization**? Какие проблемы обучения нейронной сети она решает?
7. Объясните принцип работы Dropout. Почему Dropout отключается в режиме инференса?
8. Что представляют собой логиты? В чём разница между функциями Softmax и LogSoftmax?
9. Запишите формулу функции потерь перекрёстной энтропии для задачи многоклассовой классификации.
10. Опишите классический цикл обучения PyTorch: какие шаги он включает и для чего нужен вызов `optimizer.zero_grad()`?
11. В чём различие между объектами Dataset и DataLoader? Какие задачи решает DataLoader?
12. Что такое аугментация данных? Почему она применяется только к обучающей, но не к тестовой выборке?
13. Что вычисляет функция `torch.argmax(output, dim=1)` применительно к выходу классификационной CNN?
14. Перечислите различия между методами сохранения модели через `torch.save(model, ...)` и `torch.save(model.state_dict(), ...)`. Какой из них предпочтительнее и почему?
15. Что делает контекстный менеджер `torch.no_grad()` и зачем он нужен при инференсе?
16. Какие оптимизаторы поддерживает PyTorch (модуль `torch.optim`)? В каких случаях стоит выбирать SGD с моментом, а в каких — Adam?
17. Что такое **переобучение** (overfitting)? Перечислите не менее четырёх практических способов борьбы с ним.
18. Чем характеризуется набор данных CIFAR-10: число классов, размер изображений, объёмы обучающей и тестовой выборки?
19. Почему перед подачей на сеть необходимо нормировать изображения и согласовывать порядок каналов (BGR $\rightarrow$ RGB)?
20. В каком случае целесообразно использовать **Transfer Learning**, а в каком — обучение «с нуля»? Какие слои предобученной модели обычно «замораживают»?
21. Почему сервис классификации в данной лабораторной работе реализован в виде Singleton? Какие проблемы возникнут, если загружать модель при каждом запросе?
22. Опишите путь, который проходят данные от HTTP-запроса POST /classify/ до возвращаемого JSON-ответа. Перечислите ключевые модули проекта в порядке вызова.

Содержание отчёта

Отчёт по лабораторной работе должен содержать:

1. Титульный лист.
2. Цель и задачи работы.
3. Краткое теоретическое описание сверточных нейронных сетей и фреймворка PyTorch.

-
4. Описание выполненных шагов практической части со скриншотами Swagger UI и логов обучения.
 5. Графики `train_loss` и `val_accuracy` по эпохам (для основной модели и моделей из заданий 2–3, если выполнены).
 6. Результаты выполнения заданий для самостоятельной работы.
 7. Ответы на контрольные вопросы.
 8. Список использованной литературы.

Список литературы

1. **Гудфеллоу Я., Бенджио Й., Курвилль А.** Глубокое обучение. — М.: ДМК-Пресс, 2018. — 652 с.
2. **Стивенс Э., Антига Л., Виман Т.** PyTorch. Освещаая глубокое обучение. — СПб.: Питер, 2022. — 576 с.
3. **Любанович Б.** FastAPI. Веб-разработка на Python. — СПб.: Питер, 2024.
4. **LeCun Y., Bottou L., Bengio Y., Haffner P.** Gradient-Based Learning Applied to Document Recognition // Proceedings of the IEEE. — 1998. — Vol. 86, № 11. — P. 2278–2324.
5. **Krizhevsky A., Sutskever I., Hinton G.** ImageNet Classification with Deep Convolutional Neural Networks // NeurIPS. — 2012.
6. **He K., Zhang X., Ren S., Sun J.** Deep Residual Learning for Image Recognition // CVPR. — 2016.
7. Официальная документация PyTorch — <https://pytorch.org/docs/stable/index.html>.
8. Официальная документация FastAPI — <https://fastapi.tiangolo.com/>.
9. Описание набора данных CIFAR-10 — <https://www.cs.toronto.edu/~kriz/cifar.html>.
10. Официальная документация torchvision — <https://pytorch.org/vision/stable/index.html>.